Lung Segmentation in Computed Tomography Imaging Using Deep Learning Techniques

Andrea Baraldi MAT. 839452, Sirya Caiazza MAT. 928159

Abstract—This project focuses on the segmentation of lungs in computed tomography (CT) images using deep learning techniques. The dataset employed is the LCTSC dataset from the 2017 Lung Segmentation Challenge. The raw data underwent a preprocessing pipeline that included volume cropping, resizing, intensity range normalization, and removal of slices not containing lung tissue. The processed data were then organized into a PyTorch-compatible dataset class.
Three neural network architectures, U-Net, U-Net++, and MA-Net, were trained and evaluated in both 2D and 3D configurations. A comprehensive comparative analysis of their performance was conducted. Finally, the results are discussed, highlighting potential explanations for the observed performance differences and outlining the limitations of the proposed approach.

Keywords—*lung segmentation, UNet, MAnet, 3D segmentation, DICE loss, CT images*

1. INTRODUCTION

The objective of this project is the segmentation of lung regions from computed tomography (CT) scans. To address this task, three different neural network architectures were investigated:

- **U-Net:** One of the most widely used fully convolutional neural networks for biomedical image segmentation, characterized by a symmetric encoder–decoder architecture [1].
- **U-Net++:** An extension of the original U-Net that improves segmentation accuracy through the use of nested skip connections, which help preserve fine-grained spatial information during the decoding process [6].
- **MA-Net:** The "Multi-Attention Network", which incorporates both mixed attention and self-attention mechanisms to enhance feature representation and improve segmentation performance [3].

Each architecture was implemented in both 2D and 3D configurations, resulting in a total of six trained models. All networks were implemented using the `segmentation_models_pytorch` and `segmentation_models_pytorch_3D` libraries.

The experiments were conducted using the LCTSC dataset [2], which includes 36 training volumes and 24 validation volumes, corresponding to a training-to-validation ratio of 3:2. Model evaluation was performed on a hidden test set given separately consisting of 10 volumes.

The results obtained demonstrate a favorable trade-off between segmentation accuracy and computational memory requirements.

2. DATA AND METHODOLOGY

2.1. Data Exploration

The dataset used in this study is the 2017 Lung CT Segmentation Challenge (LCTSC) dataset. It consists of a total of 60 CT volumes, which are already divided into training and validation subsets. Each CT volume has an in-plane resolution of 512×512 pixels, while the number of slices (z-dimension) varies between approximately 100 and more than 270, as illustrated in Figure 2.

Each CT volume is provided in DICOM format and includes metadata such as patient sex and age. Additionally, a ground truth segmentation mask is available for each volume. The ground truth masks identify five anatomical regions of interest:

- Left lung
- Right lung
- Esophagus
- Heart
- Spinal cord

However, for the purpose of this project, only the left and right lung regions were considered during training.

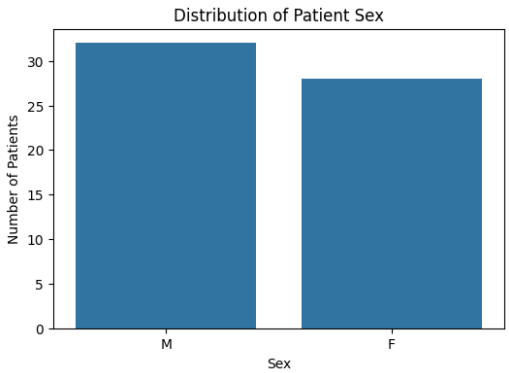

Figure 1. Distribution of patient sex across the CT volumes.

During data integrity verification, it was observed that several slices within the CT volumes did not contain lung tissue. An example of such a slice is shown in Figure 3. These slices were partially removed from the dataset to reduce computational cost and improve training efficiency.

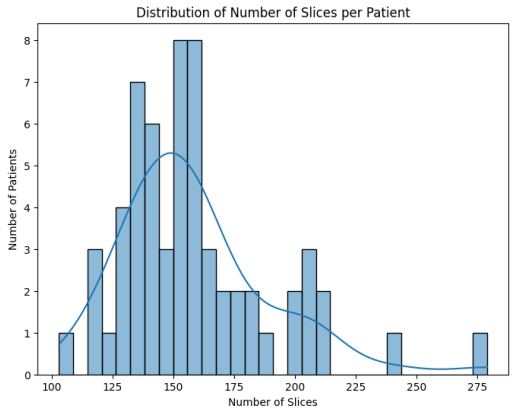

Figure 2. Distribution of the number of slices across CT volumes.

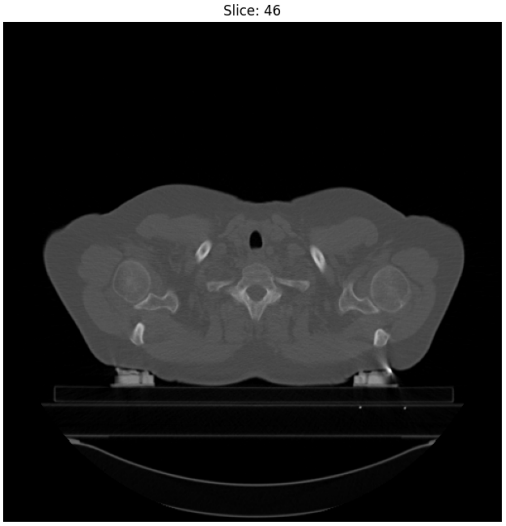

Figure 3. Example of a CT slice without visible lung tissue.

2.2. Data Preprocessing

To enable the use of CT volumes as input for the selected neural networks, a preprocessing pipeline was implemented based on the `lctsc_preprocessor.ipynb` notebook [5]. The applied transformations are described below:

- **Detection of non-zero lung masks:** All mask slices were examined to identify the slices containing non-zero values corresponding to either the left or right lung regions.
- **Coordinate extraction:** Using the slices containing lung tissue, six bounding coordinates were computed. For each spatial dimension (x, y, z), the minimum and maximum indices were determined. This allows removal of irrelevant regions while ensuring that no lung information is lost.
- **Padding:** The computed bounding coordinates were expanded by adding a 20% margin in each spatial direction. This prevents the cropping operation from being too close to anatomical boundaries and preserves contextual information.
- **Cropping:** Both CT volumes and corresponding segmentation masks were cropped using the padded bounding coordinates.
- **Windowing:** Intensity windowing was applied to the Hounsfield Unit (HU) values. Only the range $(-1000, 400)$ was retained, where -1000 HU corresponds to air and values between 300 and 400 HU approximately represent dense lung-related tissues. Values outside this range were clipped. Although a narrower lung window is standard for radiologic visualization, we opted for a wider range (-1000, 400 HU) to preserve tissue contrast in the chest wall. This strategy aims to provide the neural network with richer spatial context, enabling it to learn anatomical boundaries based on structural features (e.g., rib cage, spine) rather than relying solely on pixel intensity thresholds.
- **Normalization:** Pixel intensities were normalized to the range $[0, 1]$ to stabilize neural network training.
- **Resizing:** The spatial resolution of each slice was resized to 128×128 pixels in the x and y dimensions, while preserving the original number of slices along the z dimension. The same transformation was applied to the corresponding segmentation masks.
- **Saving:** The preprocessed volumes were saved into separate directories for training and validation data, preserving the original dataset split.

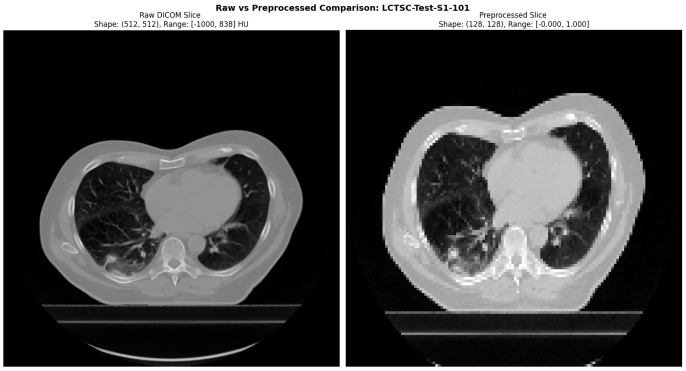

Figure 4. Comparison between a raw CT slice (left) and the corresponding preprocessed slice (right).

2.3. PyTorch Classes Implementation

Two different PyTorch dataset classes were implemented: one for processing data for the two-dimensional networks and one for the three-dimensional configuration.

The 2D dataset class was designed to create a list containing file paths and their corresponding images. Each 3D CT volume was decomposed into individual slices, which were then converted from `NumPy` arrays into

`torch.float` tensors. Finally, an additional channel dimension was added to each slice to allow PyTorch's `DataLoader` to batch the data into batches of 32 samples.

For the 3D model implementation, a similar overall design was adopted, with one key modification driven by computational constraints. Processing full 3D volumes is computationally expensive and exceeded the available hardware capacity. Therefore, instead of using entire volumes, the network was trained using randomly sampled sub-volumes. At the beginning of each training epoch, the dataset class extracts a random $64 \times 64 \times 32$ patch from each CT volume and uses it as input to the network. Since all models were trained for at least 20 epochs, this stochastic sampling strategy ensured that, over time, the network was exposed to most anatomical regions within each volume. This approach significantly reduced training time while preserving segmentation performance. A snippet of the 3D dataset class is shown in Code 1. Due to memory constraints, the batch size for the 3D models was limited to two sub-volumes.

```
class LCTSC_Dataset_3D(Dataset):
    def __init__(self, data_dir, patch_size=(64,
    ↪  64, 32), transform=None):
        [...]
    def __getitem__(self, idx):
        [...]
        # Random patch extraction
        H, W, D = image.shape
        pH, pW, pD = self.patch_size
        max_h = max(0, H - pH)
        [...]
        h = np.random.randint(0, max_h + 1) if
        ↪  max_h > 0 else 0
        [...]
        image_patch = image[h:h+pH, w:w+pW,
        ↪  d:d+pD]
        mask_patch = mask[h:h+pH, w:w+pW,
        ↪  d:d+pD]
        # Pad if necessary
        if image_patch.shape !=
        ↪  self.patch_size:
            pad_h = pH - image_patch.shape[0]
            pad_w = pW - image_patch.shape[1]
            pad_d = pD - image_patch.shape[2]
            image_patch = np.pad(image_patch,
            ↪  ((0, pad_h), (0, pad_w), (0,
            ↪  pad_d)))
        [...]
        return image_tensor, mask_tensor
```

Code 1. Patch extraction procedure for 3D model training

For both dataset implementations, it was observed that a significant portion of the initial training time was spent loading images from disk. To mitigate this bottleneck, all images were loaded into RAM during dataset initialization. This strategy leverages the faster data transfer rate between system RAM and GPU VRAM, thereby improving training efficiency.

2.4. Data Augmentation

To improve model generalization, the same data augmentation strategy was applied to both the 2D and 3D training datasets. For 2D images, the `Albumentations` library was used, while for 3D volumes the `MONAI` library was adopted, as `Albumentations` does not support full volumetric transformations. The applied augmentations are listed below; the parameter **P** denotes the probability of applying each transformation:

- **Horizontal and vertical flipping (P = 50%):** The image is flipped along the horizontal and/or vertical axes to increase spatial variability.
- **Rotation (P = 50%):** The image is rotated clockwise or counter-clockwise by up to 15° around the longitudinal (body) axis.
- **Gaussian noise (P = 30%):** Additive Gaussian noise is introduced, with a standard deviation ranging between 3% and 10% of the maximum image intensity.
- **Brightness and contrast adjustment (P = 30%):** Image brightness and contrast are independently scaled by up to ±10% to simulate acquisition variability.
- **Gaussian blur (P = 20%):** The image is smoothed using a Gaussian filter with a kernel standard deviation randomly sampled between 3 and 5.

3. TRAINING AND RESULTS

3.1. Training Configuration

For all six models, the ResNet34 encoder was selected to ensure a fair comparison across architectures, and all network weights were initialized randomly. A comparison of the model sizes is reported in Figure 5 for the 2D architectures and in Figure 6 for the 3D architectures.

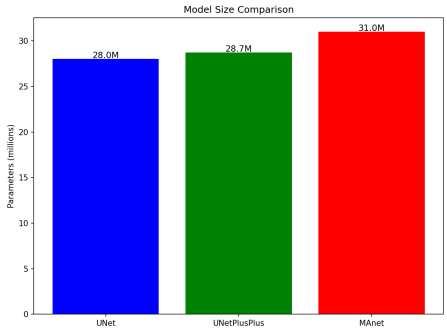

Figure 5. Number of trainable parameters for the 2D models.

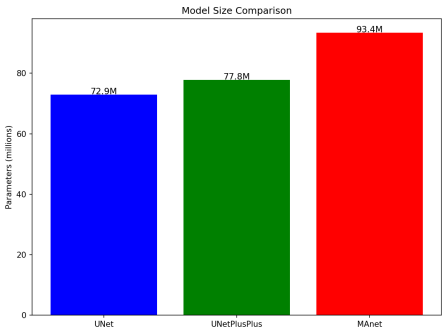

Figure 6. Number of trainable parameters for the 3D models.

All models were trained using the configuration reported in Code 2.

```
# Hardware configuration
CPU: Ryzen 7 5800x
RAM: 32 GB DDR4 3600 MHz
GPU: Nvidia RTX 3070 8GB

# Training configuration
NUM_EPOCHS = 50
EARLY_STOPPING_PATIENCE = 10
EARLY_STOPPING_MIN_DELTA = 0.001
```

Code 2. Training and hardware configuration

As the loss function, the `DiceLoss` implementation provided by the respective libraries was used, since the primary objective of the task was to maximize the Dice similarity coefficient. Training was performed using an early stopping strategy, where training was terminated if the validation loss did not improve for 10 consecutive epochs. The model achieving the lowest validation loss was retained for evaluation.

The `Adam` optimizer was used with a learning rate of 1×10^{-4}. Additionally, a learning rate scheduler based on PyTorch's `ReduceLROnPlateau` was implemented. The scheduler reduced the learning rate by a factor of two whenever the validation loss did not improve for five consecutive epochs. This strategy is commonly adopted to allow finer optimization once the training process approaches convergence.

3.2. Results

To obtain a comprehensive evaluation of model performance, four quantitative metrics were computed: Dice coefficient, Intersection over Union (IoU), Sensitivity, and Precision [4].

- **Dice coefficient:** Measures the overlap between the predicted segmentation and the ground truth. It is defined as twice the overlapping area divided by the total number of pixels in both masks.
- **IoU:** Also known as the Jaccard index, it measures the ratio between the intersection area and the union area of the predicted and ground truth masks.
- **Sensitivity (Recall):** Measures the proportion of correctly identified positive pixels and is defined as the ratio between true positives and all ground truth positive pixels.
- **Precision:** Measures the proportion of correctly predicted positive pixels and is defined as the ratio between true positives and all predicted positive pixels.

Compared to the Dice coefficient, IoU penalizes oversegmentation and under-segmentation more strongly. Therefore, high values for both Dice and IoU indicate high-quality segmentation results. Sensitivity is particularly informative for medical image segmentation tasks because it focuses on correctly identifying anatomical structures even in highly imbalanced images where background pixels dominate.

The quantitative results for the validation dataset are reported in Table 1 for the 2D models and Table 2 for the 3D models.

Model	Dice	IoU	Sensitivity	Precision	Best val loss
UNet	0.910 ± 0.165	0.861 ± 0.181	0.905	0.945	0.054
UNet++	0.930 ± 0.140	0.888 ± 0.157	0.923	0.958	0.045
MANet	0.929 ± 0.154	0.892 ± 0.169	0.930	0.956	0.044

Table 1. Results for the models in the 2D configuration.

Model	Dice	IoU	Sensitivity	Precision	Best val loss
UNet	0.970 ± 0.010	0.936 ± 0.019	**0.976**	0.958	0.043
UNet++	0.966 ± 0.018	0.934 ± 0.032	0.975	0.958	0.045
MANet	**0.972 ± 0.011**	**0.945 ± 0.021**	0.972	**0.972**	**0.036**

Table 2. Results for the models in the 3D configuration.

3.3. Evaluation and Comparison

The results highlight two main observations. First, the 3D models consistently achieved higher performance across all evaluation metrics. Although the improvement in mean Dice and IoU scores is moderate, the most significant advantage of the 3D models lies in their substantially lower standard deviation. For all architectures, the variance in performance was approximately one order of magnitude smaller than in the 2D case, indicating more stable and consistent segmentation results.

This stability is clearly visible in the validation loss curves shown in Figures 9 and 10, where the oscillations observed in the 3D models are considerably reduced compared to the 2D models. The improved stability can be attributed to the ability of 3D networks to exploit volumetric spatial context, which is particularly beneficial for anatomical structure segmentation.

The second observation concerns architectural differences. In the 2D configuration, the MA-Net architecture achieved slightly better performance than both U-Net and

U-Net++. This improvement is likely due to the attention mechanisms embedded in MA-Net, which enhance feature selection and improve spatial focus during segmentation. Moreover, MA-Net exhibited the most stable validation loss among the 2D models, as shown in Figure 9.

In contrast, the performance differences between architectures in the 3D configuration were minimal. This suggests that the volumetric spatial information exploited by 3D convolutions reduces the relative advantage of attention mechanisms.

Another notable observation is that the 2D U-Net triggered early stopping earlier than the other 2D architectures, as shown in Figure 9. This behaviour is likely related to its simpler skip-connection structure, which may limit the network's ability to efficiently recover fine spatial details compared to U-Net++ and MA-Net.

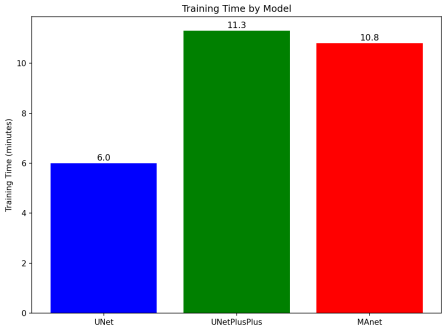

Figure 7. Training time (minutes) for the 2D models.

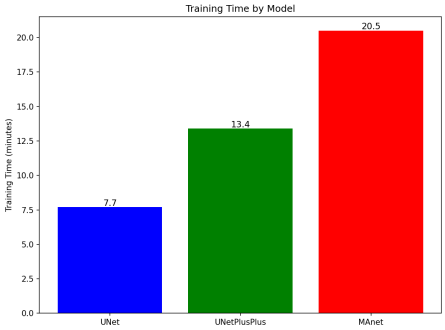

Figure 8. Training time (minutes) for the 3D models.

The differences in training behaviour are further confirmed by the validation loss trends, which were used as the early stopping criterion. The 2D U-Net shows the largest oscillations in validation loss, indicating less stable convergence.

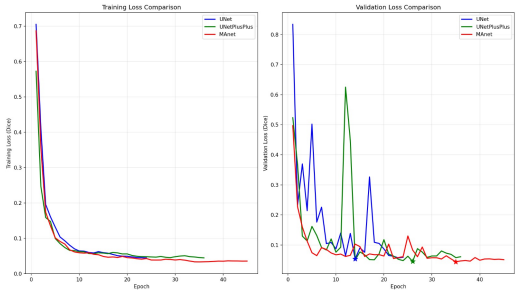

Figure 9. Training loss (left) and validation loss (right) for the 2D models.

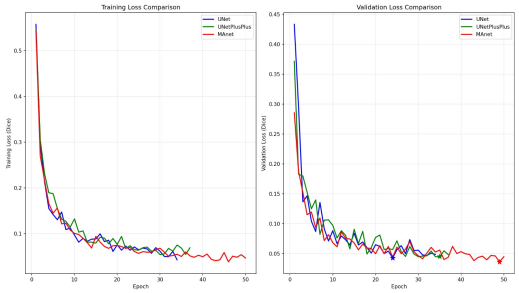

Figure 10. Training loss (left) and validation loss (right) for the 3D models.

We further assessed the models' usability in a realistic deployment setting by measuring the inference time on a single volume. The benchmark was repeated ten times, and the average inference time was reported.

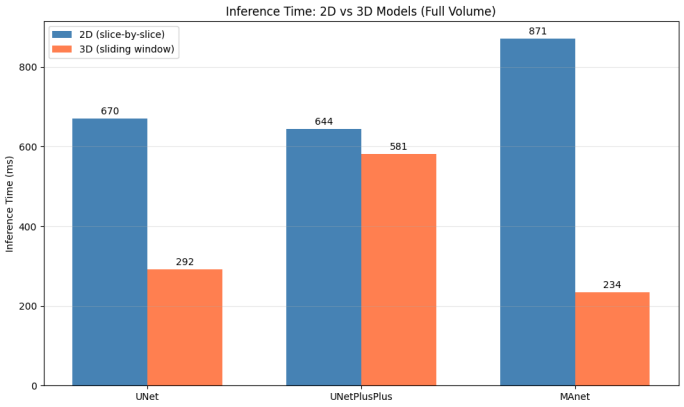

Figure 11. Inference times for one volume

Once again, the 3D MANet resulted to be the best model, achieving the lowest inference time despite being the most computationally heavy, highlighting its efficiency. Overall, 3D models consistently outperform 2D models in terms of speed, likely due to the sliding-window strategy, which processes an entire 3D volume by dividing it into smaller overlapping patches and appears to be more efficient than slice-by-slice inference.

4. DISCUSSION

Based on the quantitative results presented in the previous section, the model selected for testing was the 3D MA-Net architecture. This choice was motivated by its consistently superior performance across most evaluation metrics. Although the 3D MA-Net has a slightly larger number of parameters and requires marginally longer training times compared to the other architectures, it achieved the highest overall segmentation accuracy. Furthermore, its performance variability, measured through standard deviation across metrics, was either lower than or comparable to that of the other models, indicating strong robustness and reliability.

The results suggest that combining volumetric spatial context with attention mechanisms allows the network to better capture anatomical structures and improve segmentation quality. The 3D convolutions enable the model to exploit inter-slice correlations, while the attention modules enhance feature representation and spatial localization, leading to more precise predictions.

4.1. Segmentation masks postprocessing and testing

Because the test images lacked ground truth, we inspected 3D renderings of the predicted segmentations to obtain an empirical assessment of model performance. Although the model commonly recovered the overall lung shape, it produced spurious predictions near the patient's body. To investigate this behaviour we generated a heatmap of the CT Hounsfield unit (HU) distribution.

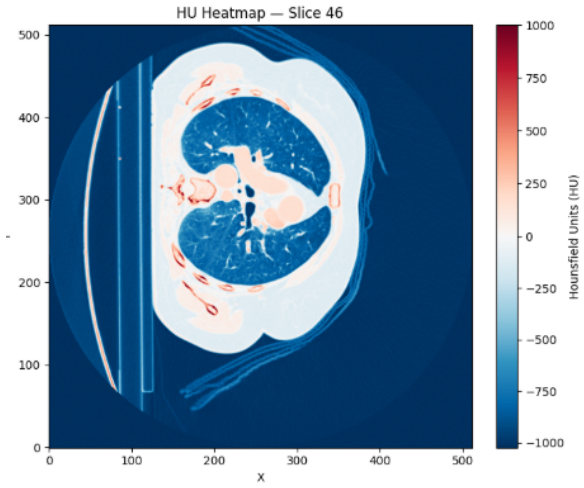

Figure 12. Distribution of the HU units of a central slice of the body

The heatmap revealed that portions of the patient blanket and the examination table exhibited HU values similar to aerated lung tissue; this caused the model to sometimes classify those regions as lung. We also observed occasional inclusion of tracheal air in the lung mask, which is an expected type of error.

To remove body-adjacent artifacts, we built a binary body mask from the normalized CT volume and applied it to the predicted segmentation. The procedure was as follows:

1. Threshold the normalize image to exclude air outside the patient (i.e., select intensity values consistent with tissue and table rather than external air).
2. Fill holes inside the resulting body contour (for example, the lungs and airways) so that the body silhouette becomes a single solid region.
3. Label connected components and retain only the largest component, which corresponds to the patient's body; this step removes small floating artifacts and table edges.
4. Produce a binary mask where 1 indicates voxels inside the body and 0 indicates outside (air/background), and multiply this mask with the predicted lung segmentation.

The final segmentation is therefore constrained to the interior of the patient body, which reduces false positives caused by the blanket, table, and other external artifacts.

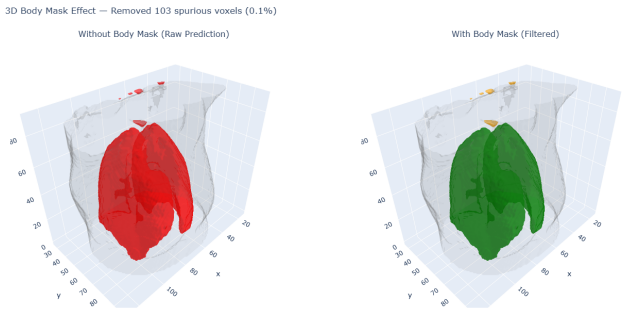

Figure 13. Effect of postprocessing on generated segmentation mask. In orange the segments removed by the postprocessing.

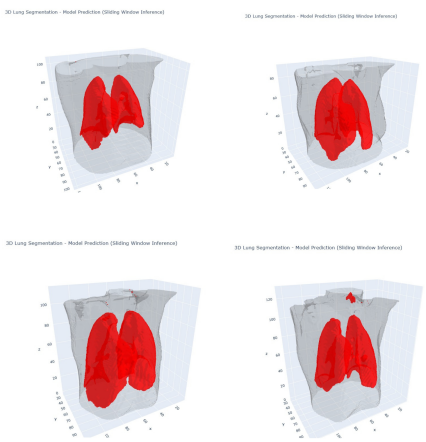

Figure 14. Some postprocessed masks generated by the 3D MANet on test images.

5. CONCLUSIONS AND FUTURE WORK

This project addressed the task of lung segmentation in CT images using deep learning techniques. The dataset employed was the 2017 Lung CT Segmentation Challenge dataset [2]. The raw data underwent a preprocessing pipeline that included cropping, resizing, intensity windowing, and normalization to improve data consistency and training efficiency.

Three segmentation architectures, U-Net, U-Net++, and MA-Net, were evaluated in both 2D and 3D configurations. Among all tested models, the 3D MA-Net demonstrated the best overall performance, achieving a Dice coefficient of 0.972 ± 0.011, an IoU score of 0.945 ± 0.021, and a validation loss of 0.036. This model was therefore selected for testing, and the predicted segmentation masks were further refined using a light post-processing step to improve accuracy.

Future work could focus on extending this study by leveraging more powerful computational resources. Increased hardware capacity would allow training 3D models using larger batch sizes and higher-resolution volumetric inputs, which may further improve segmentation stability and accuracy. Additionally, future investigations could explore more advanced data augmentation strategies, alternative attention-based architectures, and multiclass segmentation approaches that simultaneously identify additional anatomical structures. Finally, evaluating model generalization on external datasets would provide further insight into the robustness and clinical applicability of the proposed methodology.

REFERENCES

[1] O. Ronneberger, P. Fischer, and T. Brox, *U-net: Convolutional networks for biomedical image segmentation*, 2015. arXiv: 1505.04597 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1505.04597.

[2] G. S. e. a. J. Yang, *Data from lung ct segmentation challenge (lctsc) (version 3) [data set]. the cancer imaging archive.* 2017. [Online]. Available: https://doi.org/10.7937/K9/TCIA.2017.3R3FVZ08.

[3] M. Jian, N. Yang, and C. Zhu, "Manet: Multi-attention network for polyp segmentation", *Medical Engineering Physics*, vol. 143, p. 104 396, 2025, ISSN: 1350-4533. DOI: https://doi.org/10.1016/j.medengphy.2025.104396. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1350453325001158.

[4] N. Huynh. "Understanding evaluation metrics in medical image segmentation". (), [Online]. Available: https://medium.com/mastering-data-science/understanding-evaluation-metrics-in-medical-image-segmentation-d289a373a3f.

[5] I. Postuma. "Lctscproject2024". (), [Online]. Available: https://github.com/AI-For-Experimental-And-Applied-Physics/LCTSCProject2024/tree/main/lctsc_preprocessor.

[6] A. Taparia. "Unet++ architecture explained". (), [Online]. Available: https://www.geeksforgeeks.org/machine-learning/unet-architecture-explained/.